PROJECT B: DATA QUERY BUILDER

PGE3 Workshop - Practical Project Day 2

Automating SQL workflows through Model Context Protocol (MCP)

The Vision

Bridging the gap between natural language questions and structured data retrieval.

The "Language-to-Data" Challenge

The Problem

Non-technical stakeholders often struggle to write complex SQL queries, leading to bottlenecks in data-driven decision-making and constant reliance on engineering teams.

Our Solution

An MCP server that enables AI agents to autonomously load CSVs, understand schemas, and execute read-only SQL queries via conversational interfaces.

Technological Architecture

Python FastMCP

High-performance framework for building scalable Model Context Protocol servers.

SQLite

Reliable, lightweight in-memory database engine for dynamic table management.

Claude / Gemini

Advanced LLMs acting as the orchestrator for tool invocation and reasoning.

MCP Tool Implementation Matrix

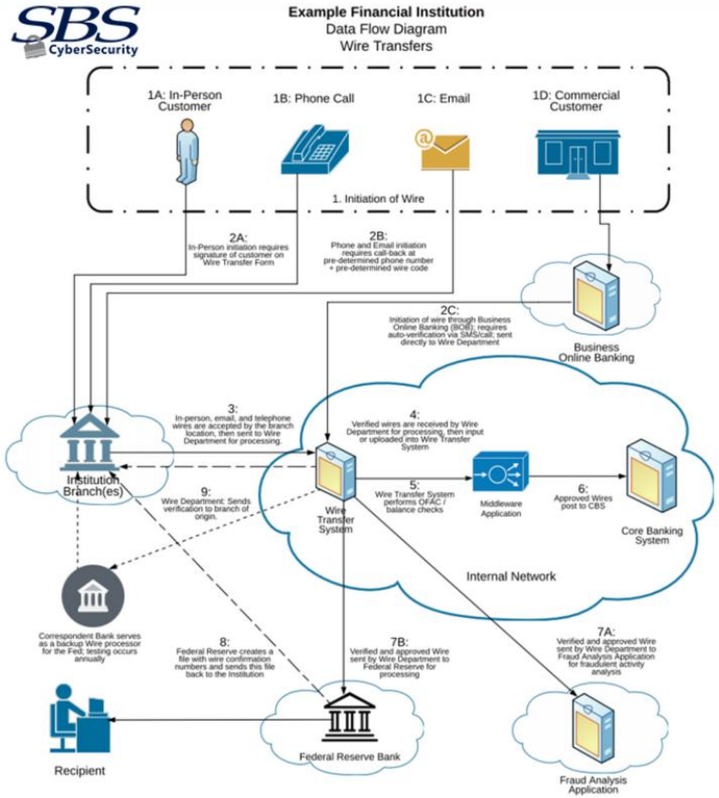
Tool Name	Functionality	Key Parameters
load_csv	Import CSV into SQLite table with auto-types	file_path, table_name
describe_schema	List tables and their columns/types	(None)
run_query	Execute read-only SQL queries safely	sql, limit (default 50)
get_statistics	Compute count, mean, min, max for columns	table_name, column
list_tables	View all tables and row counts	(None)

Security & Integrity Constraints

We implemented a robust **Security Guardrail** in the `security.py` module to prevent unauthorized data manipulation:

- 🔒 **Read-Only Mode:** Strictly enforces SELECT statements.
- 🚫 **Keyword Rejection:** Block DELETE, DROP, INSERT, and UPDATE.
- 🛡️ **Error Handling:** Returns helpful security messages to the agent instead of crashing.

Data Retrieval Pipeline



1. Ingestion

CSV Loaded via SQLite Helper

2. Reasoning

LLM calls describe_schema

3. Execution

run_query fetches specific rows

4. Synthesis

Natural Language result delivery

Implementation Metrics

05

Specialized Tools

Modular Codebase

The repository is organized following professional software engineering patterns:

- `src/tools/`: Atomic tool implementations.
- `src/sqlite_helper.py`: Abstracted database logic.
- `src/security.py`: Centralized safety validation.

This structure ensures **High Cohesion** and **Low Coupling**.

Agent Prompting Strategies

⚡ Minimal Strategy

Claude identifies tools via docstrings. Best for simple one-off questions like "What tables are in my database?"

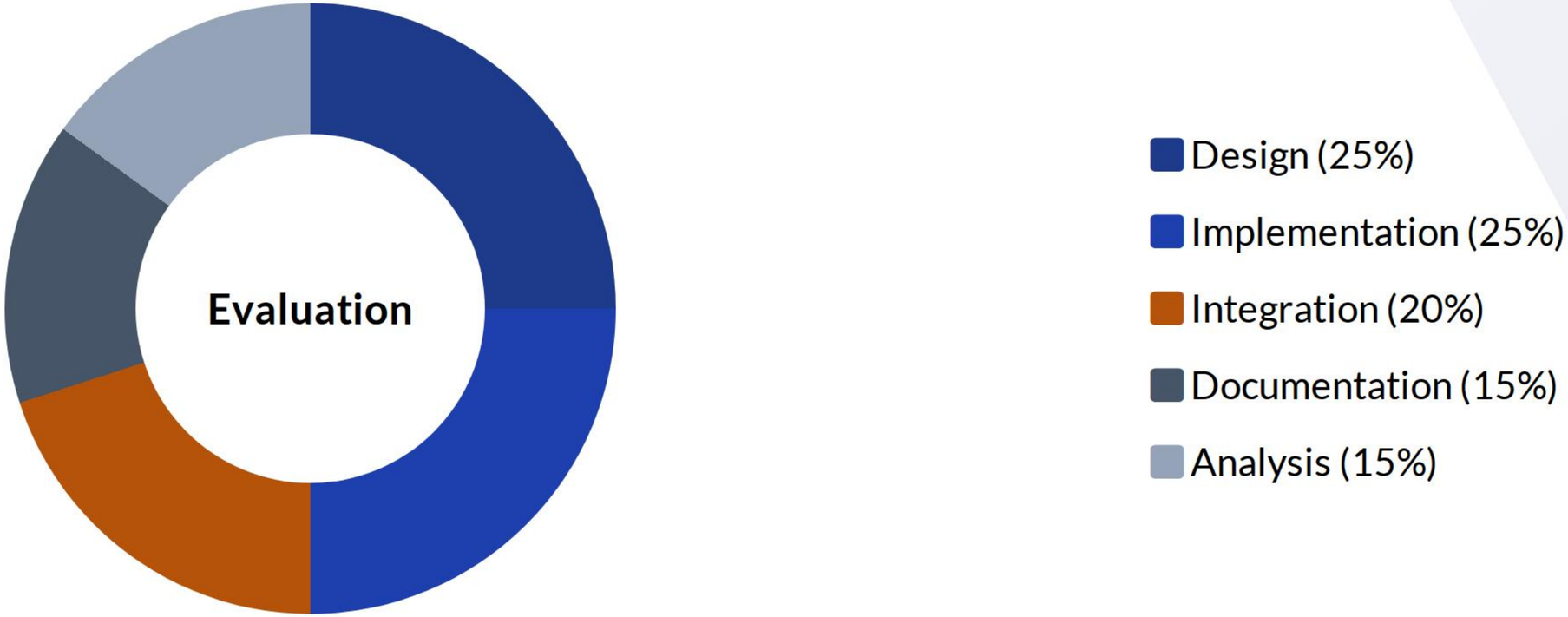
🧠 Expert Workflow

A multi-phase prompt instructing the agent to first map the schema, then plan the SQL, and finally verify results before reporting.

Strategy 3 (Expert) significantly improves accuracy for multi-table joins.

Comparative Value Analysis

Scoring & Grading Breakdown



Questions?

Project B: Data Query Builder MCP Server

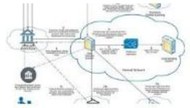
/data-query-builder-mcp

Workshop Complete | ReAct Loop | FastMCP | Python 3.10+

Image Sources

https://static.vecteezy.com/system/resources/previews/060/007/221/non_2x/security-shield-cyber-security-protection-shield-with-check-mark-icon-in-wireframe-hand-cyber-data-protection-information-privacy-idea-network-safety-circuit-board-pattern-illustration-vector.jpg

Source: www.vecteezy.com



https://sbscyber.com/hubfs/Images/BlogImages/DFD.png

Source: sbscyber.com

---

https://png.pngtree.com/thumb_back/fw800/background/20250226/pngtree-modern-office-workspace-featuring-a-computer-with-data-analytics-on-the-image_16965319.jpg

Source: pngtree.com